

Paris 1 Panthéon-Sorbonne University
U.F.R. 27 – Mathematics and Computer Science

RAPPORT DE PROJET
PROBABILITÉS, SIMULATION ET THÉORIE DES GRAPHS
PROJET ACADÉMIQUE

THÈME

Percolation par arêtes : Simulation, probabilité de croisement et dualité

Réalisé par :

— Marc SU

Axes du projet :

— Percolation par arêtes
— Simulation Monte Carlo
— Dualité planaire

Année universitaire 2025 – 2026

Résumé exécutif

Ce rapport présente une étude de la percolation par arêtes sur une grille finie. Chaque arête est déclarée ouverte indépendamment avec une probabilité p , et l'objectif principal consiste à déterminer s'il existe un chemin ouvert reliant le bord gauche au bord droit de la grille.

Le projet combine trois dimensions complémentaires :

- une modélisation probabiliste fondée sur des variables aléatoires de Bernoulli indépendantes ;
- une simulation informatique utilisant une exploration de graphe par parcours en largeur ;
- une analyse théorique de la probabilité de croisement $\theta_n(p)$, incluant la croissance en fonction de p , des bornes probabilistes et un argument de dualité planaire.

La partie numérique estime la probabilité de croisement par simulation Monte Carlo pour plusieurs tailles de grilles. Les résultats mettent en évidence une transition rapide autour de $p = 1/2$, ce qui est cohérent avec le seuil critique connu pour la percolation par arêtes sur le réseau carré.

Table des matières

Résumé exécutif	1
1 Description du modèle et simulation	4
1.1 Percolation par arêtes sur une grille finie	4
1.2 Clusters et événement de croisement	5
1.3 Représentation graphique	5
1.4 Workflow de simulation	5
2 Probabilité de croisement	7
2.1 Définition de $\theta_n(p)$	7
2.2 Croissance de la probabilité de croisement	7
2.3 Estimation Monte Carlo	8
2.4 Table de résultats numériques	9
2.5 Interprétation des résultats numériques	10
3 Bornes et comportement critique	11
3.1 Une borne par comptage des chemins	11
3.2 Probabilité critique	12
3.3 Interprétation du seuil critique	12
4 Dualité planaire	14
4.1 Intuition du graphe dual	14
4.2 Figure de dualité	14
4.3 Complémentarité des croisements	15
5 Implémentation algorithmique	16
5.1 Parcours en largeur	16
5.2 Complexité	16
5.3 Choix d'implémentation	17
6 Validation, limites et extensions	18
6.1 Validation de la simulation	18
6.2 Limites du projet	18
6.3 Extensions possibles	18

6.4	Intérêt professionnel	19
6.5	Alternative algorithmique : Union-Find	19
6.6	Compétences développées	19
Conclusion		20
A Code Python		21
Bibliographie		23

Chapitre 1

Description du modèle et simulation

1.1 Percolation par arêtes sur une grille finie

On considère une grille carrée finie de taille n . L'ensemble des sommets est défini par

$$V_n = \{0, 1, \dots, n\} \times \{0, 1, \dots, n\}.$$

Deux sommets sont reliés par une arête lorsque leur distance euclidienne est égale à 1. L'ensemble des arêtes est noté E_n .

Définition 1.1: Percolation par arêtes

Une configuration de percolation par arêtes sur une grille finie est un sous-ensemble aléatoire d'arêtes. Chaque arête $e \in E_n$ est déclarée ouverte avec probabilité $p \in [0, 1]$ et fermée avec probabilité $1 - p$, indépendamment de toutes les autres arêtes.

De manière équivalente, on associe à chaque arête e une variable aléatoire de Bernoulli

$$X_e = \begin{cases} 1, & \text{si l'arête } e \text{ est ouverte,} \\ 0, & \text{si l'arête } e \text{ est fermée.} \end{cases}$$

Alors

$$X_e \sim \mathcal{B}(p).$$

Proposition 1.1: Nombre d'arêtes ouvertes

Soit M_n le nombre d'arêtes ouvertes dans la grille. Alors

$$M_n = \sum_{e \in E_n} X_e.$$

Comme les variables $(X_e)_{e \in E_n}$ sont indépendantes et suivent une loi de Bernoulli de paramètre p , on a

$$M_n \sim \mathcal{B}(|E_n|, p).$$

En particulier,

$$\mathbb{E}[M_n] = |E_n|p, \quad \text{Var}(M_n) = |E_n|p(1 - p).$$

Démonstration

Le nombre d'arêtes ouvertes est la somme des indicatrices X_e . Chaque X_e suit une loi de Bernoulli de paramètre p , et la famille est indépendante. Par définition, une somme de variables de Bernoulli indépendantes de même paramètre suit une loi binomiale. Les formules de l'espérance et de la variance sont donc les formules classiques de la loi binomiale.

1.2 Clusters et événement de croisement

Définition 1.2: Cluster

Un cluster est une composante connexe du graphe formé par les arêtes ouvertes. Autrement dit, deux sommets appartiennent au même cluster s'il existe un chemin composé uniquement d'arêtes ouvertes qui les relie.

Définition 1.3: Croisement gauche-droite

On dit qu'il existe un croisement gauche-droite lorsqu'il existe un chemin ouvert reliant au moins un sommet du bord gauche

$$\{0, 1, \dots, n\} \times \{0\}$$

à au moins un sommet du bord droit

$$\{0, 1, \dots, n\} \times \{n\}.$$

L'événement de croisement est noté

$$C_n = \{\text{il existe un chemin ouvert de gauche à droite}\}.$$

1.3 Représentation graphique

Les figures suivantes illustrent trois configurations de percolation obtenues pour différentes valeurs de p . Les arêtes ouvertes appartenant au cluster exploré depuis le bord gauche sont mises en évidence. On observe que le comportement global change fortement lorsque p augmente.

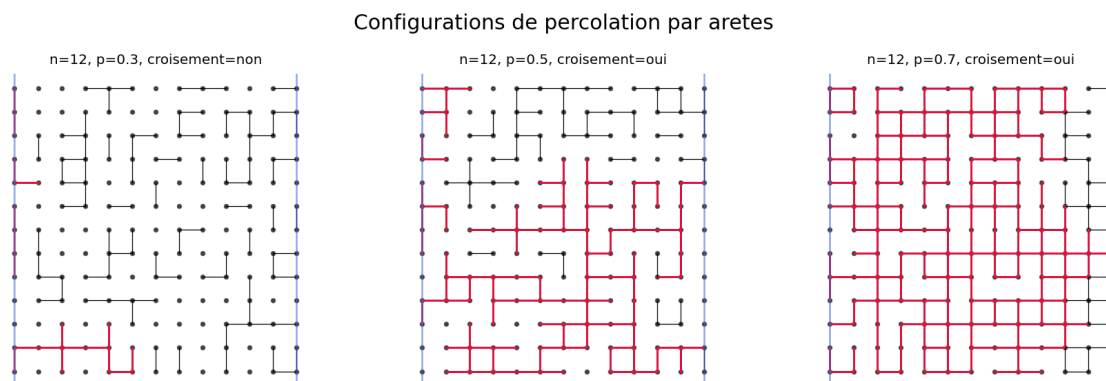


FIGURE 1.1 – Exemples de configurations de percolation par arêtes pour différentes valeurs de p .

1.4 Workflow de simulation

La simulation peut être décomposée en cinq étapes principales.

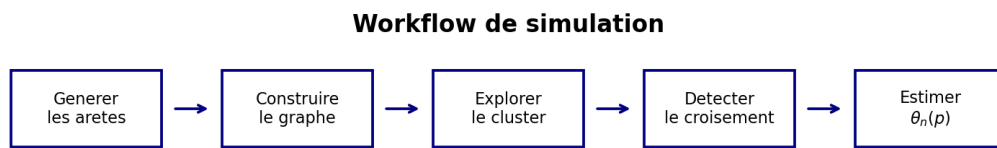


FIGURE 1.2 – Workflow utilisé pour simuler le modèle de percolation et estimer la probabilité de croisement.

1. Générer des variables de Bernoulli indépendantes pour toutes les arêtes.
2. Construire le graphe induit par les arêtes ouvertes.
3. Lancer une exploration à partir de tous les sommets du bord gauche.
4. Utiliser un parcours en largeur pour identifier tous les sommets connectés au bord gauche.
5. Vérifier si l'exploration atteint le bord droit.

Chapitre 2

Probabilité de croisement

2.1 Définition de $\theta_n(p)$

Définition 2.1: Probabilité de croisement

Pour $n \in \mathbb{N}^*$ et $p \in [0, 1]$, on définit

$$\theta_n(p) = \mathbb{P}(C_n),$$

où C_n désigne l'événement selon lequel le bord gauche est relié au bord droit par un chemin ouvert.

La fonction

$$p \longmapsto \theta_n(p)$$

mesure donc la probabilité d'observer un croisement dans une grille finie de taille n .

2.2 Croissance de la probabilité de croisement

Théorème 2.1: Croissance de $\theta_n(p)$

Pour tout $n \in \mathbb{N}^*$ fixé, la fonction

$$p \longmapsto \theta_n(p)$$

est croissante sur $[0, 1]$.

Démonstration

Soit $(U_e)_{e \in E_n}$ une famille de variables aléatoires indépendantes suivant une loi uniforme sur $[0, 1]$. Pour une probabilité p , on déclare l'arête e ouverte si

$$U_e \leq p.$$

Si $p \leq p'$, alors toute arête ouverte au niveau p est aussi ouverte au niveau p' , car

$$U_e \leq p \leq p'.$$

Ainsi, lorsque l'on passe de p à p' , on peut ouvrir de nouvelles arêtes, mais on ne ferme aucune arête déjà ouverte. Donc tout croisement gauche-droite existant pour p existe encore pour p' . On a donc

$$C_n(p) \subset C_n(p').$$

Par conséquent,

$$\theta_n(p) = \mathbb{P}(C_n(p)) \leq \mathbb{P}(C_n(p')) = \theta_n(p').$$

La fonction θ_n est donc croissante.

2.3 Estimation Monte Carlo

La probabilité $\theta_n(p)$ peut être estimée numériquement par répétition d'expériences indépendantes. Si N_{sim} configurations sont générées et si Y_i désigne l'indicatrice d'un croisement pour la i -ième simulation, alors l'estimateur Monte Carlo est

$$\hat{\theta}_n(p) = \frac{1}{N_{\text{sim}}} \sum_{i=1}^{N_{\text{sim}}} Y_i.$$

Proposition 2.1: Estimateur sans biais

L'estimateur Monte Carlo $\hat{\theta}_n(p)$ est sans biais :

$$\mathbb{E}[\hat{\theta}_n(p)] = \theta_n(p).$$

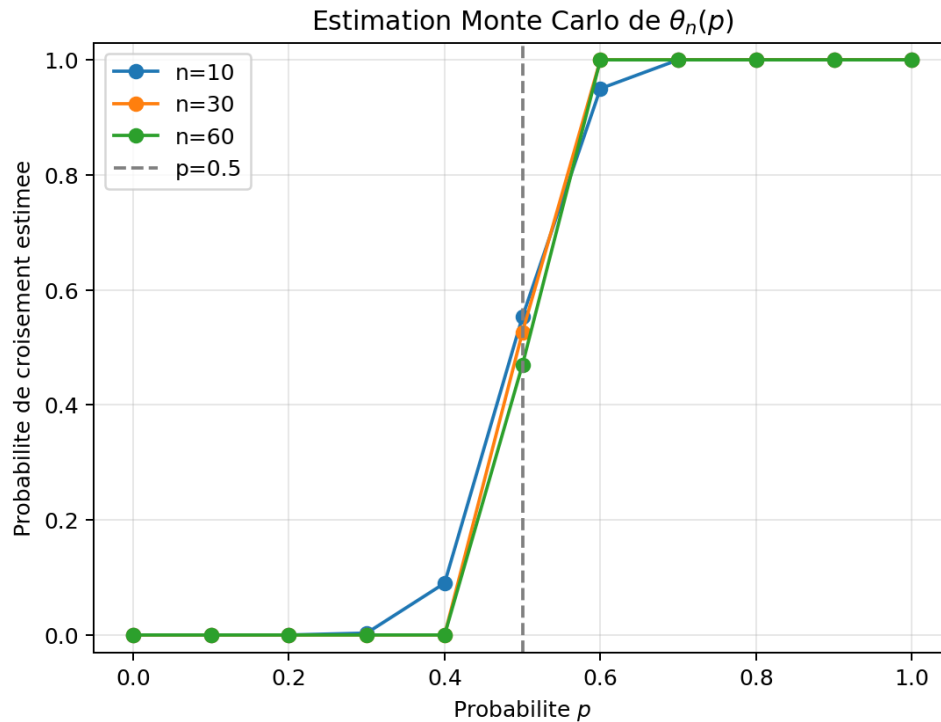
Démonstration

Comme Y_i est l'indicatrice de l'événement C_n , on a

$$\mathbb{E}[Y_i] = \mathbb{P}(C_n) = \theta_n(p).$$

Donc

$$\mathbb{E}[\hat{\theta}_n(p)] = \mathbb{E} \left[\frac{1}{N_{\text{sim}}} \sum_{i=1}^{N_{\text{sim}}} Y_i \right] = \frac{1}{N_{\text{sim}}} \sum_{i=1}^{N_{\text{sim}}} \mathbb{E}[Y_i] = \theta_n(p).$$

FIGURE 2.1 – Estimation Monte Carlo de $\theta_n(p)$ pour différentes tailles de grille.**Proposition 2.2: Variance et intervalle de confiance**

Comme Y_i suit une loi de Bernoulli de paramètre $\theta_n(p)$, la variance de l'estimateur Monte Carlo est

$$\text{Var}(\hat{\theta}_n(p)) = \frac{\theta_n(p)(1 - \theta_n(p))}{N_{\text{sim}}}.$$

Un intervalle de confiance approximatif à 95% est donné par

$$\hat{\theta}_n(p) \pm 1.96 \sqrt{\frac{\hat{\theta}_n(p)(1 - \hat{\theta}_n(p))}{N_{\text{sim}}}}.$$

2.4 Table de résultats numériques

Le tableau suivant donne un exemple de résultats obtenus par simulation Monte Carlo avec $N_{\text{sim}} = 1000$ répétitions pour chaque couple (n, p) . Les valeurs peuvent légèrement varier selon la graine aléatoire utilisée. Les valeurs du tableau ne doivent pas nécessairement coïncider exactement avec celles de la figure, car elles dépendent du nombre de simulations, de la grille de valeurs de p et de la graine aléatoire utilisée.

Taille n	$p = 0.4$	$p = 0.5$	$p = 0.6$
10	0.104	0.542	0.935
30	0.003	0.538	0.999
60	0.000	0.518	1.000

TABLEAU 2.1 – Estimations de $\theta_n(p)$ par simulation Monte Carlo avec $N_{\text{sim}} = 1000$.

2.5 Interprétation des résultats numériques

Les résultats numériques mettent en évidence un phénomène de transition. Pour $p = 0.4$, la probabilité de croisement devient très faible lorsque la taille n augmente. Cela signifie que les chemins ouverts restent généralement locaux et ne parviennent pas à relier les deux bords opposés.

Pour $p = 0.6$, la probabilité de croisement devient presque égale à 1 dès que la grille est suffisamment grande. Dans ce régime, les arêtes ouvertes sont assez nombreuses pour former un grand cluster traversant la grille.

La zone la plus intéressante se situe autour de $p = 0.5$. Pour cette valeur, les simulations donnent une probabilité proche de 0.5, ce qui traduit un équilibre entre l'existence d'un chemin ouvert gauche-droite et l'existence d'un chemin dual haut-bas. Lorsque n augmente, la transition entre le régime sans croisement et le régime avec croisement devient de plus en plus abrupte.

Remarque 2.1: Interprétation statistique

Le nombre de simulations contrôle la précision statistique, tandis que la taille n contrôle l'échelle géométrique du problème. Augmenter N_{sim} réduit le bruit Monte Carlo. Augmenter n rend la transition autour de la probabilité critique plus nette.

Chapitre 3

Bornes et comportement critique

3.1 Une borne par comptage des chemins

Une manière naturelle de majorer la probabilité de croisement consiste à compter les chemins ouverts possibles. Si un croisement gauche-droite existe, alors il existe au moins un chemin ouvert partant du bord gauche et de longueur au moins n .

Proposition 3.1: Borne de l'union pour les croisements

Soit N_k un majorant du nombre de chemins de longueur k partant du bord gauche. Sur le réseau carré, on peut utiliser le majorant grossier

$$N_k \leq (n+1)4^k.$$

Cette borne est volontairement grossière : elle autorise des chemins avec retours en arrière et compte donc davantage de chemins que nécessaire. Elle suffit cependant pour obtenir une majoration simple.

Alors

$$\theta_n(p) \leq \sum_{k \geq n} (n+1)4^k p^k.$$

Démonstration

Pour un chemin fixé de longueur k , la probabilité que toutes ses arêtes soient ouvertes vaut p^k , par indépendance. Un croisement implique l'existence d'au moins un chemin ouvert de longueur au moins n partant du bord gauche. Par la borne de l'union,

$$\theta_n(p) \leq \sum_{k \geq n} N_k p^k.$$

En utilisant $N_k \leq (n+1)4^k$, on obtient

$$\theta_n(p) \leq \sum_{k \geq n} (n+1)(4p)^k.$$

Théorème 3.1: Disparition de la probabilité de croisement pour p petit

Si $p < 1/4$, alors

$$\lim_{n \rightarrow \infty} \theta_n(p) = 0.$$

Démonstration

Si $p < 1/4$, alors $4p < 1$. D'après la borne précédente,

$$\theta_n(p) \leq (n+1) \sum_{k \geq n} (4p)^k = (n+1) \frac{(4p)^n}{1-4p}.$$

Comme $(4p)^n$ décroît exponentiellement vite, ce terme domine le facteur polynomial $n+1$. Ainsi,

$$(n+1)(4p)^n \longrightarrow 0.$$

Donc

$$\theta_n(p) \longrightarrow 0.$$

3.2 Probabilité critique

Définition 3.1: Probabilité critique

La probabilité critique p_c est la valeur qui sépare le régime sous-critique, où les longues connexions ouvertes sont peu probables, du régime surcritique, où les longues connexions deviennent probables.

Pour la percolation par arêtes sur le réseau carré infini, il est connu que

$$p_c = \frac{1}{2}.$$

Dans le cadre fini, cette valeur apparaît numériquement comme le point autour duquel la probabilité de croisement passe rapidement de valeurs proches de 0 à des valeurs proches de 1.

Remarque 3.1: Effet de taille finie

Pour une taille n finie, la transition n'est pas parfaitement brutale. La courbe $p \mapsto \theta_n(p)$ est croissante et relativement lisse. Cependant, lorsque n augmente, la zone de transition devient plus étroite, ce qui donne visuellement l'impression d'un seuil de plus en plus net.

3.3 Interprétation du seuil critique

Les simulations suggèrent que la valeur critique est proche de $1/2$. Ce n'est pas un hasard : pour la percolation par arêtes sur le réseau carré, le seuil critique exact est

$$p_c = \frac{1}{2}.$$

L'idée intuitive repose sur l'auto-dualité du réseau carré. Lorsque $p = 1/2$, les arêtes ouvertes et les arêtes fermées jouent des rôles symétriques.

Théorème 3.2: Seuil critique du réseau carré

Pour la percolation de Bernoulli par arêtes sur le réseau carré infini $\mathbb{Z}^2$, la probabilité critique est

$$p_c = \frac{1}{2}.$$

Démonstration

Une preuve complète de ce théorème dépasse le cadre de ce rapport, car elle nécessite des résultats plus profonds de la théorie de la percolation en dimension deux.

Dans ce rapport, l'argument de dualité sur un rectangle fini est utilisé pour motiver la valeur $p_c = 1/2$. La preuve rigoureuse sur le réseau infini demande des outils supplémentaires de théorie de la percolation. Néanmoins, l'idée centrale est que, pour $p = 1/2$, le modèle primal et le modèle dual ont la même loi. Les croisements ouverts gauche-droite et les croisements duaux haut-bas sont alors en équilibre.

Chapitre 4

Dualité planaire

4.1 Intuition du graphe dual

Dans la percolation planaire, le graphe dual est obtenu en plaçant un sommet dual dans chaque face du graphe initial, puis en dessinant des arêtes duales qui traversent les arêtes du graphe primal. Une arête duale est ouverte exactement lorsque l'arête primale correspondante est fermée.

Cela crée une relation complémentaire entre les chemins ouverts dans le graphe primal et les chemins ouverts dans le graphe dual. Dans un rectangle fini, s'il n'existe pas de croisement ouvert gauche-droite dans le graphe primal, alors il existe un croisement dual haut-bas.

Définition 4.1: Configuration duale

À partir d'une configuration de percolation de paramètre p , la configuration duale est obtenue en déclarant une arête duale ouverte lorsque l'arête primale correspondante est fermée. Ainsi, les arêtes duales sont ouvertes avec probabilité $1 - p$.

4.2 Figure de dualité

La figure suivante illustre le principe de dualité. Les arêtes pleines représentent le graphe primal, tandis que les arêtes pointillées représentent le graphe dual. Un chemin primal gauche-droite empêche l'existence simultanée d'un chemin dual haut-bas, car les deux chemins devraient nécessairement se couper.

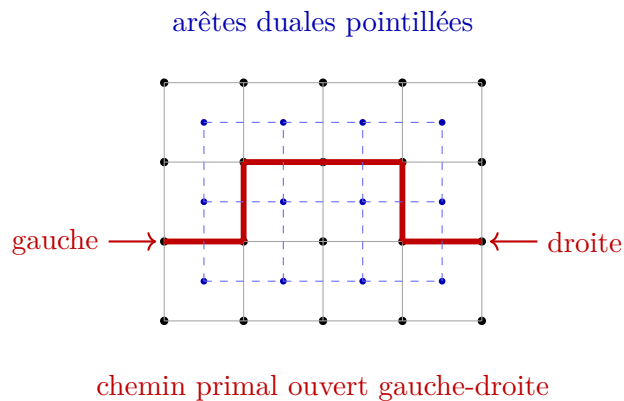


FIGURE 4.1 – Illustration du graphe primal et du graphe dual dans une grille planaire. Les arêtes rouges représentent un chemin primal ouvert. Les arêtes bleues pointillées représentent le graphe dual. Un croisement primal gauche-droite bloque nécessairement un croisement dual haut-bas.

4.3 Complémentarité des croisements

Théorème 4.1: Identité de dualité dans un rectangle fini

Pour une grille rectangulaire adaptée et son dual, l'événement de croisement primal gauche-droite est complémentaire de l'événement de croisement dual haut-bas. Par conséquent,

$$\theta_n(p) + \theta_n(1 - p) = 1.$$

Démonstration

La preuve repose sur la planarité. Un chemin primal ouvert traversant le rectangle de gauche à droite empêche tout chemin dual ouvert de traverser le rectangle de haut en bas, car les deux chemins devraient se croiser. À un point d'intersection, une arête primale et son arête duale correspondante ne peuvent pas être ouvertes simultanément, puisque l'arête duale est ouverte exactement lorsque l'arête primale est fermée.

Réciproquement, s'il n'existe pas de chemin primal ouvert de gauche à droite, on peut suivre la frontière séparant le cluster connecté au bord gauche du reste du rectangle. Cette frontière forme un chemin ouvert dans le graphe dual reliant le haut au bas.

Ainsi, exactement l'un des deux événements suivants se produit :

- un croisement ouvert gauche-droite dans le graphe primal ;
- un croisement ouvert haut-bas dans le graphe dual.

Comme le graphe dual a pour paramètre $1 - p$, la probabilité du second événement est $\theta_n(1 - p)$. On obtient donc

$$\theta_n(p) + \theta_n(1 - p) = 1.$$

Remarque 4.1: Pourquoi $p = 1/2$ est particulier

Si $p = 1/2$, alors $1 - p = p$. L'identité de dualité donne

$$\theta_n(1/2) + \theta_n(1/2) = 1,$$

donc

$$\theta_n(1/2) = \frac{1}{2}.$$

Cela explique pourquoi la probabilité de croisement observée empiriquement est proche de 0.5 autour de $p = 0.5$.

Chapitre 5

Implémentation algorithmique

5.1 Parcours en largeur

L'objectif informatique est de déterminer s'il existe un chemin ouvert reliant le bord gauche au bord droit. Il s'agit d'un problème d'exploration de graphe.

Définition 5.1: Parcours en largeur

Le parcours en largeur, ou *Breadth-First Search* (BFS), est un algorithme qui explore un graphe couche par couche. À partir d'un ensemble de sommets initiaux, il visite d'abord leurs voisins, puis les voisins de ces voisins, et ainsi de suite.

Dans le problème de percolation, le BFS démarre depuis tous les sommets du bord gauche et ne se déplace qu'à travers les arêtes ouvertes. Si l'exploration atteint le bord droit, alors il existe un croisement gauche-droite.

Workflow algorithmique

1. Générer indépendamment toutes les arêtes horizontales et verticales avec probabilité p .
2. Initialiser une file avec tous les sommets du bord gauche.
3. Explorer les sommets voisins uniquement à travers les arêtes ouvertes.
4. Marquer chaque sommet visité afin d'éviter les boucles et les répétitions.
5. Si un sommet du bord droit est atteint, retourner **True**.
6. Si la file devient vide, retourner **False**.

5.2 Complexité

Soient $|V_n|$ le nombre de sommets et $|E_n|$ le nombre d'arêtes. Le parcours en largeur explore chaque sommet et chaque arête ouverte au plus une fois. Sa complexité est donc

$$O(|V_n| + |E_n|).$$

Pour une grille $n \times n$,

$$|V_n| = (n + 1)^2, \quad |E_n| = 2n(n + 1).$$

La complexité est donc d'ordre

$$O(n^2).$$

5.3 Choix d'implémentation

L'implémentation évite les difficultés liées au *splicing* en séparant les arêtes horizontales et verticales :

- les arêtes horizontales sont stockées dans une matrice de taille $(n + 1) \times n$;
- les arêtes verticales sont stockées dans une matrice de taille $n \times (n + 1)$.

Remarque 5.1: Amélioration pratique

Au lieu de forcer toutes les arêtes dans une seule matrice artificielle, séparer les arêtes horizontales et verticales rend le code plus clair, réduit les erreurs d'indexation et améliore la lisibilité de la simulation.

Chapitre 6

Validation, limites et extensions

6.1 Validation de la simulation

Plusieurs vérifications permettent de s'assurer que l'implémentation est cohérente :

- lorsque $p = 0$, aucune arête n'est ouverte, donc la probabilité de croisement doit être nulle ;
- lorsque $p = 1$, toutes les arêtes sont ouvertes, donc la probabilité de croisement doit être égale à 1 ;
- lorsque p augmente, la probabilité de croisement estimée doit globalement augmenter, à de petites fluctuations Monte Carlo près ;
- autour de $p = 1/2$, la probabilité de croisement doit être proche de $1/2$ pour des rectangles symétriques.

Remarque 6.1: Fiabilité numérique

Les méthodes Monte Carlo sont aléatoires : deux exécutions peuvent donner des valeurs légèrement différentes. Pour rendre les résultats reproductibles, l'implémentation utilise une graine aléatoire. Pour améliorer la précision, on peut augmenter le nombre de simulations.

6.2 Limites du projet

Le modèle étudié est volontairement simple. Il se concentre sur une percolation par arêtes indépendante sur une grille régulière finie. Ce cadre est riche mathématiquement, mais présente plusieurs limites :

- seules des grilles finies sont simulées, alors que la transition de phase théorique est un phénomène de volume infini ;
- les arêtes sont supposées indépendantes, ce qui n'est pas toujours réaliste dans des applications de réseaux réels ;
- la simulation se concentre sur les croisements gauche-droite et n'analyse pas entièrement la distribution des tailles de clusters ;
- la convergence vers le comportement du réseau infini est seulement observée numériquement.

6.3 Extensions possibles

Plusieurs prolongements pourraient enrichir le projet :

- étudier la percolation par sites, où ce sont les sommets et non les arêtes qui sont ouverts ou fermés ;

- estimer la distribution de la taille du plus grand cluster ;
- calculer la taille moyenne des clusters en fonction de p ;
- comparer le parcours en largeur avec un algorithme Union-Find ;
- réaliser des simulations Monte Carlo plus larges pour observer le *finite-size scaling* ;
- étendre la simulation à d'autres graphes, comme les réseaux triangulaires, hexagonaux ou aléatoires.

6.4 Intérêt professionnel

Les modèles de percolation dépassent le cadre de la théorie des probabilités. Ils apparaissent en science des réseaux, en épidémiologie, en science des matériaux, en analyse de fiabilité et en physique statistique. Ils permettent d'étudier comment des connexions locales aléatoires peuvent produire une connectivité globale, une fragmentation ou une transition de phase.

Ce projet illustre donc un workflow complet : définir un modèle mathématique, implémenter une simulation, estimer une probabilité numériquement, valider les résultats et les interpréter à l'aide d'outils théoriques.

6.5 Alternative algorithmique : Union-Find

Une autre approche possible consiste à utiliser une structure de données Union-Find. À chaque arête ouverte, on fusionne les deux composantes contenant ses extrémités. Il suffit ensuite de vérifier si une composante connecte un sommet du bord gauche à un sommet du bord droit. Cette méthode est particulièrement efficace lorsque l'on souhaite répéter de nombreuses simulations, suivre l'évolution des composantes connexes ou analyser la taille des clusters.

Dans le cadre de ce rapport, le parcours en largeur a été privilégié car il est simple à interpréter et directement relié à la recherche d'un chemin ouvert. L'algorithme Union-Find constitue néanmoins une extension naturelle pour rendre la simulation plus efficace à grande échelle.

6.6 Compétences développées

Ce projet a permis de mobiliser plusieurs compétences importantes :

- modélisation probabiliste à partir de variables de Bernoulli ;
- simulation Monte Carlo et estimation numérique d'une probabilité ;
- exploration de graphes avec un parcours en largeur ;
- interprétation d'un phénomène de transition de phase ;
- implémentation Python structurée et reproductible ;
- validation numérique et analyse critique des limites du modèle.

Conclusion

Ce projet a étudié la percolation par arêtes à travers une approche à la fois théorique et computationnelle. Le modèle a été défini à l'aide de variables aléatoires de Bernoulli indépendantes attachées aux arêtes d'une grille finie. L'événement principal étudié était l'existence d'un chemin ouvert gauche-droite, dont la probabilité est notée $\theta_n(p)$.

La partie théorique a permis d'établir plusieurs propriétés importantes. Premièrement, $\theta_n(p)$ est croissante par rapport à p . Deuxièmement, un argument de comptage des chemins fournit une borne montrant que les croisements deviennent improbables pour des valeurs suffisamment petites de p . Enfin, la dualité planaire donne l'identité centrale

$$\theta_n(p) + \theta_n(1 - p) = 1,$$

qui explique le rôle particulier de $p = 1/2$.

Les simulations numériques confirment le phénomène de transition attendu. Pour des petites probabilités, les croisements sont rares ; pour des grandes probabilités, ils deviennent presque certains ; et autour de $p = 1/2$, la probabilité de croisement est proche de $1/2$. Lorsque la taille de la grille augmente, la transition devient plus nette.

Ainsi, ce projet montre comment la théorie des probabilités, les algorithmes de graphes et la simulation numérique peuvent être combinés pour analyser un modèle mathématique ayant une structure géométrique riche.

Chapitre A

Code Python

Le code Python suivant implémente la simulation de la percolation par arêtes, détecte les croisements gauche-droite par parcours en largeur et estime la probabilité de croisement par simulation Monte Carlo.

```
1 import numpy as np
2 from collections import deque
3
4
5 def generate_bond_percolation(n, p, rng=None):
6     """
7     Genere une configuration de percolation par aretes sur une grille n x n.
8
9     Les sommets sont indexes par (i, j), avec i, j dans {0, ..., n}.
10
11     horizontal[i, j] represente l'arete entre (i, j) et (i, j+1).
12     vertical[i, j] represente l'arete entre (i, j) et (i+1, j).
13     """
14     if rng is None:
15         rng = np.random.default_rng()
16
17     horizontal = rng.random((n + 1, n)) < p
18     vertical = rng.random((n, n + 1)) < p
19
20     return horizontal, vertical
21
22
23 def has_left_right_crossing(horizontal, vertical):
24     """
25     Verifie s'il existe un chemin ouvert du bord gauche
26     vers le bord droit en utilisant un parcours en largeur.
27     """
28     n = horizontal.shape[1]
29     visited = np.zeros((n + 1, n + 1), dtype=bool)
30     queue = deque()
31
32     # Initialisation avec tous les sommets du bord gauche
33     for i in range(n + 1):
34         visited[i, 0] = True
35         queue.append((i, 0))
36
37     while queue:
38         i, j = queue.popleft()
39
40         # Si le bord droit est atteint, un croisement existe
41         if j == n:
42             return True
43
44         # Deplacement vers la gauche
45         if j > 0 and horizontal[i, j - 1] and not visited[i, j - 1]:
46             visited[i, j - 1] = True
47             queue.append((i, j - 1))
48
49         # Deplacement vers la droite
50         if j < n and horizontal[i, j] and not visited[i, j + 1]:
51             visited[i, j + 1] = True
52             queue.append((i, j + 1))
53
```

```

54     # Déplacement vers le haut
55     if i > 0 and vertical[i - 1, j] and not visited[i - 1, j]:
56         visited[i - 1, j] = True
57         queue.append((i - 1, j))
58
59     # Déplacement vers le bas
60     if i < n and vertical[i, j] and not visited[i + 1, j]:
61         visited[i + 1, j] = True
62         queue.append((i + 1, j))
63
64     return False
65
66
67 def estimate_theta(n, p, simulations=1000, seed=42):
68     """
69     Estime theta_n(p), la probabilité d'un croisement gauche-droite,
70     par simulation Monte Carlo.
71     """
72     rng = np.random.default_rng(seed)
73     crossings = 0
74
75     for _ in range(simulations):
76         horizontal, vertical = generate_bond_percolation(n, p, rng)
77         if has_left_right_crossing(horizontal, vertical):
78             crossings += 1
79
80     return crossings / simulations
81
82
83 def confidence_interval(theta_hat, simulations, level=1.96):
84     """
85     Renvoie un intervalle de confiance approximatif à 95% pour theta_hat.
86     """
87     standard_error = np.sqrt(theta_hat * (1 - theta_hat) / simulations)
88     lower = max(0.0, theta_hat - level * standard_error)
89     upper = min(1.0, theta_hat + level * standard_error)
90     return lower, upper
91
92
93 if __name__ == "__main__":
94     sizes = [10, 30, 60]
95     probabilities = [0.4, 0.5, 0.6]
96     simulations = 1000
97
98     for n in sizes:
99         print(f"n = {n}")
100         for p in probabilities:
101             theta_hat = estimate_theta(n, p, simulations=simulations)
102             ci_low, ci_high = confidence_interval(theta_hat, simulations)
103             print(
104                 f"p = {p:.2f}, theta_hat = {theta_hat:.3f}, "
105                 f"95% CI = [{ci_low:.3f}, {ci_high:.3f}]"
106             )
107         print("-" * 60)

```

Listing A.1 – Simulation de la percolation par arêtes en Python

Bibliographie

1. G. Grimmett, *Percolation*, Springer, 1999.
2. H. Kesten, *Percolation Theory for Mathematicians*, Birkhäuser, 1982.
3. B. Bollobás and O. Riordan, *Percolation*, Cambridge University Press, 2006.